

La función `print()` → entrada/salida

- Comportamiento base: `print()` envía datos a la consola. Si se invoca sin argumentos, imprime una línea en blanco.
- Múltiples argumentos: se preparan por comas. `print()` los emite en una sola línea añadiendo automáticamente un espacio entre ellos.
- Argumentos de palabra clave (keyword arguments): se deben colocar después de los argumentos posicionales.
 - `sep`: Define el separador entre argumentos (por defecto es un espacio " ").
 - `end`: Define que se imprime al final de la declaración (por defecto es un salto de línea "\n").
- La función `input()`: obtiene datos introducidos por el usuario. Siempre devuelve una cadena (str), por lo que nos puede usar directamente en operaciones matemáticas sin antes convertir su tipo. Puede recibir un mensaje como argumento para mostrarlo en pantalla.

Tipos de Datos Literales

Un literal son datos cuyos valores están determinados por el propio literal.

- Enteros (int): Números sin parte fraccionaria.
 - soportan guiones bajos para mejorar la legibilidad (ej. 11_111_111)
 - base octal: prefijo 0o o 0O (ej. 0o123).
 - base hexadecimal: prefijo 0x o 0X (ej. 0x123.)
- Punto Flotante (float): Números con parte decimal no vacía o representados en notación científica.
 - se utiliza el punto . como separador decimal.
 - notación científica: usa e o E para indicar "por diez a la potencia" (ej. 3E0 equivale a 3×10^0).
- Cadenas (str): Texto delimitado por comillas dobles ("...") o simples ('...').

- Caracter de escape (\): La barra invertida le da un significado especial al siguiente caracter:
 - \n: Nueva línea
 - \r: Inicializa una barra invertida
 - \" o \' : Escapa comillas dentro de una cadena delimitada por el mismo tipo de comilla.
- Booleanos (bool): Representan valores de verdad: True (1) y False (0). Tienen sintaxis estricta con mayúscula inicial.
- None: Literal especial de tipo NoneType que representa la ausencia de un valor.

Conversión de tipos (casting) y operadores de cadenas

- Funciones de conversión:
 - int(x): Convierte el argumento x a un número entero
 - float(x): Convierte el argumento x a un número flotante
 - str(x): Convierte el número o dato x en una cadena.
- Operadores aplicados a cadenas:
 - Concatenación (+): Junta dos cadenas (cadena1 + cadena2). Ambos operandos deben ser cadenas.
 - Replicación (*): Multiplica una cadena n veces (cadena * entero). Un número ≤ 0 genera una cadena vacía.

Variables y Reglas de Nomenclatura

- Creación: una variable se crea automáticamente en el momento en que se le asigna un valor con el operador =. No requiere declaración previa.
- Tipo dinámico: El tipo de la variable cambia según el valor asignado.
- Reglas para nombres (identificadores):
 - Deben comenzar con una letra o un guión bajo (_).
 - Pueden contener letras, dígitos y guiones bajos.
 - Es sensible a mayúsculas y minúsculas (var y VAR son diferentes).
 - No pueden ser palabras reservadas (if, for, class, import, etc.).
 - Convención PEP 8: usar minúsculas separadas por guiones bajos (ej: mi_variable).

[illegible]

← → ↻ 🌐 edube.org/learn/python-essentials-1-esp-edube/laboratorio-literales-de-python-cadenas-3 🔍 ☆ J ☰ | 📁 Todos los marcadores

Python INSTITUTO

2.2.1.11 LABORATORIO: Literales de Python - Cadenas

LABORATORIO

Tiempo Estimado
5-10 minutos

Nivel de Dificultad
Fácil

Objetivos

- Familiarizarse con la función `print()` y sus capacidades de formato.
- Practicar el codificar cadenas.
- Experimentar con el código de Python.

Escenario
Escribe una sola línea de código, utilizando la función `print()`, así como los caracteres de nueva línea y escape, para obtener la salida esperada de tres líneas.

Salida Esperada

```
"Esoy"
"aprendiendo"
"python"
```

```
1 print("Esoy"+"aprendiendo"+"python")
```

Console 1

```
"Esoy"
"aprendiendo"
"python"
```

Prev Next

← → ↻ 🌐 edube.org/learn/python-essentials-1-esp-edube/laboratorio-variables-2 🔍 ☆ J ☰ | 📁 Todos los marcadores

Python INSTITUTO

2.4.1.7 LABORATORIO: Variables

LABORATORIO

Tiempo Estimado
10 minutos

Nivel de Dificultad
Fácil

Objetivos

- Familiarizarse con el concepto de almacenar y trabajar con diferentes tipos de datos en Python.
- Experimentar con el código en Python.

Escenario
A continuación una historia:
Érase una vez en la Tierra de las Manzanas, Juan tenía tres manzanas, María tenía cinco manzanas, y Adán tenía seis manzanas. Todos eran muy felices y vivieron por muchísimo tiempo. Fin de la Historia.

Tu tarea es:

- Crear las variables: `juan`, `maria` y `adan`.
- Asignar valores a las variables. El valor debe de ser igual al número de manzanas que cada quien tenía.
- Una vez almacenados los números en las variables, imprimir las variables en una línea, y separar cada una de ellas con una coma.
- Después se debe crear una nueva variable llamada `total_manzanas` y se debe igualar a la suma de las tres variables anteriores.
- Imprimir el valor almacenado en `total_manzanas` en la consola.
- Experimenta con tu código: crea nuevas variables, asigna diferentes valores a ellas, y realiza varias operaciones aritméticas con ellas (por ejemplo `+`, `-`, `*`, `/`, etc.) Intenta poner una cadena con un entero juntos en la misma línea, por ejemplo, "Número Total de Manzanas:" y `total_manzanas`.

```
1 juan = 3
2 maria = 5
3 adan = 6
4 print(juan, maria, adan, sep=", ")
5
6 total_manzanas = juan + maria + adan
7 print(total_manzanas)
8
9 print("Número Total de Manzanas:", total_manzanas)
```

Console 1

```
3, 5, 6
14
Número Total de Manzanas: 14
```

Prev Next

← → ↻ 🌐 edube.org/learn/python-essentials-1-esp-edube/laboratorio-variables-un-sencillo-convertidor-2 🔍 ☆ J ☰ | 📁 Todos los marcadores

Python INSTITUTO

2.4.1.9 LABORATORIO: Variables, un sencillo convertidor

LABORATORIO

Tiempo Estimado
10 minutos

Nivel de Dificultad
Fácil

Objetivos

- Familiarizarse con el concepto de variables y trabajar con ellas.
- Realizar operaciones básicas y conversiones.
- Experimentar con el código de Python.

Escenario
Millas y kilómetros son unidades de longitud o distancia.
Teniendo en mente que 1 milla equivale aproximadamente a 1.61 kilómetros, complementa el programa en el editor para que convierta de:

- Millas a kilómetros.
- Kilómetros a millas.

No se debe cambiar el código existente. Escribe tu código en los lugares indicados con `###`. Prueba tu programa con los datos que han sido provistos en el código fuente.

Pon mucha atención a lo que está ocurriendo dentro de la función `print()`. Analiza como es que se proveen múltiples argumentos para la función, y como es que se muestra el resultado.

Nota que algunos de los argumentos dentro de la función `print()` son cadenas (por ejemplo "millas son") y otros son variables (por ejemplo `millas`).

Hay una cosa interesante más que está ocurriendo. ¿Puedes ver otra función dentro de la función `print()`? Es la función `round()`. Su trabajo es redondear la salida del resultado al número de decimales especificados en el paréntesis, y regresar un valor flotante (dentro de la función `round()` se puede encontrar el nombre de la variable, el nombre, una coma, y el número de decimales que se desean mostrar). Se hablará más de esta función muy pronto, no te preocupes si no todo queda muy claro. Solo se quiere impulsar tu curiosidad.

Después de completar el laboratorio, ahora Sandbox, y experimenta más. Intenta escribir diferentes convertidores, por ejemplo, un convertidor de USD.

```
1 kilometros = 16.09
2
3 millas = 1.61
4
5 kilometros_to_millas = kilometros / 1.61
6
7 print(millas, "millas son", round(millas_to_kilometros, 2), "kilometros")
8 print(kilometros, "kilometros son", round(kilometros_to_millas, 2), "millas")
9
```

Console 1

```
16.09 millas son 11.80 kilometros
16.09 kilometros son 10.61 millas
```

Prev Next

← → ↺

edube.org/learn/python-essentials-1-esp-edube/laboratorio-operadores-y-expresiones-6

🔍 ☆ J ⋮

Todos los marcadores

Python

2.4.1.10 LABORATORIO: Operadores y expresiones

2.4.1.10 LABORATORIO: Operadores y expresiones

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con los conceptos de números, operadores y operaciones aritméticas en Python.
- Realizar cálculos básicos.

Escenario

Observa el código en el editor: lee un valor `floatante`, lo coloca en una variable llamada `x`, e imprime el valor de la variable llamada `y`. Tu tarea es completar el código para evaluar la siguiente expresión:

$$3x^3 - 2x^2 + 3x - 1$$

El resultado debe ser asignado a `y`.

Recuerda que la notación algebraica clásica muy seguido omite el operador de multiplicación, aquí se debe de incluir de manera explícita. Nota como se cambia el tipo de dato para asegurarnos de que `y` es del tipo `floatante`.

Mantén tu código limpio y legible, y pruébalo utilizando los datos que han sido proporcionados. No te desanimes por no lograrlo en el primer intento. Se persistente y curioso.

Datos de Prueba

Entrada de Muestra

```
x = 0
x = 1
x = -1
```

Salida Esperada

```
y = -1.0
y = 2.0
y = -19.0
```

Python

2.4.1.10 LABORATORIO: Operadores y expresiones

2.4.1.10 LABORATORIO: Operadores y expresiones

```
1 # Escribe aquí tu código para asignar el valor para x: 7)
2
3 y = 3 * x**3 - 2 * x**2 + 3 * x - 1
4
5 print(y, x**3, x**2)
```

Consola

Entrada al valor para x: 2
y = 21.0

Prev

Next

← → ↺

edube.org/learn/python-essentials-1-esp-edube/laboratorio-comentarios-2

🔍 ☆ J ⋮

Todos los marcadores

Python

2.5.1.2 LABORATORIO: Comentarios

2.5.1.2 LABORATORIO: Comentarios

LABORATORIO

Tiempo Estimado

5 minutos

Nivel de Dificultad

Muy Fácil

Objetivos

- Familiarizarse con el concepto de comentarios en Python.
- Utilizar y no utilizar los comentarios.
- Reemplazar los comentarios con código.
- Experimentar con el código de Python.

Escenario

El código en el editor contiene comentarios. Intenta mejorarlo: agrega o quita comentarios donde consideres que sea apropiado (en ocasiones el remover un comentario lo hace más legible), además, cambia el nombre de las variables donde consideres que esto mejorará la comprensión del código.

NOTA

Los comentarios son muy importantes. No solo hacen que el programa sea **más fácil de entender**, pero también sirven para **deshabilitar aquellas partes de código que no son necesarias** (por ejemplo, cuando se necesita probar cierta parte del código, e ignorar el resto). Los buenos programadores **describen** todo parte importante del código, y dan **nombres significativos** a variables, debido a que en ocasiones es mucho más sencillo dejar el comentario dentro del código mismo.

Es bueno utilizar nombres de variables **legibles**, y en ocasiones es mejor **dividir el código** en partes con nombres (por ejemplo en funciones). En algunas situaciones, es una buena idea escribir los pasos de como se realizaron los cálculos de una forma sencilla y clara.

Una cosa más puede ocurrir que un comentario contenga una pieza de información incorrecta o errónea, nunca se debe de hacer eso a propósito.

Python

2.5.1.2 LABORATORIO: Comentarios

2.5.1.2 LABORATORIO: Comentarios

```
1 # Este programa calcula los segundos en un número determinado de horas preestablecidas
2
3 horas = 2 # número de horas
4 segundos_por_hora = 3600 # número de segundos en una hora
5
6 print("Segundos en horas: ", horas) # imprime el número de segundos en ese número de horas
7 print("Segundos en horas: ", horas * segundos_por_hora) # se imprime el número de segundos en ese número de horas
8
9 # Este es el fin del programa que calcula el número de segundos en 2 horas
```

Consola

Horas: 2
Segundos en horas: 7200

Prev

Next

edube.org/learn/python-essentials-1-esp-edube/laboratorio-entradas-y-salidas-simples-2

Todos los marcadores

Python INSTITUTE

2.6.1.9 LABORATORIO: Entradas y salidas simples

LABORATORIO

Tiempo Estimado

5-10 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con la entrada y salida de datos en Python.
- Evaluar expresiones simples.

Escenario

La tarea es completar el código para evaluar y mostrar el resultado de cuatro operaciones aritméticas básicas.

El resultado debe ser mostrado en consola.

Quizá no podrás proteger el código de un usuario que intente dividir entre cero. Por ahora, no hay que preocuparse por ello.

Prueba tu código - ¿Produce los resultados esperados?

No te mostraremos ningún dato de prueba, eso sería demasiado sencillo.

```
1 a = float(input("Ingresa el primer valor (a): "))
2 b = float(input("Ingresa el segundo valor (b): "))
3
4 print("Suma:", a + b)
5 print("Resta:", a - b)
6 print("Multiplicación:", a * b)
7 print("División:", a / b)
8
9 print("\n¡Eso es todo, amigos!")
```

Console

```
¡Eso es todo, amigos!
Ingresa el primer valor (a): 2
Ingresa el segundo valor (b): 3
Suma: 5.0
Resta: -1.0
Multiplicación: 6.0
División: 0.6666666666666666
¡Eso es todo, amigos!
```

Escribe aquí para buscar.

Prev Next

ESP 11:22 p. m. 23/09/2026

edube.org/learn/python-essentials-1-esp-edube/laboratorio-operadores-y-expresiones-7

Todos los marcadores

Python INSTITUTE

2.6.1.10 LABORATORIO: Operadores y expresiones

LABORATORIO

Tiempo Estimado

20 minutos

Nivel de Dificultad

Intermedio

Objetivos

- Familiarizarse con los conceptos de números, operadores y expresiones aritméticas en Python.
- Comprender la precedencia y asociatividad de los operadores de Python, así como el correcto uso de los paréntesis.

Escenario

La tarea es completar el código para poder evaluar la siguiente expresión:

$$x + \frac{1}{x + \frac{1}{x + \frac{1}{x}}}$$

El resultado debe de ser asignado a `y`. Se cauteloso, observa los operadores y priorízalos. Utiliza cuantos

```
1 x = float(input("Ingresa el valor para x: "))
2
3 y = 1 / (x + 1 / (x + 1 / (x + 1 / x)))
4
5 print("y =", y)
6
```

Console

```
Ingresa el valor para x: 5
y = 0.19258202567760344
```

Escribe aquí para buscar.

Prev Next

ESP 11:23 p. m. 23/09/2026

Python INSTITUTE

2.6.1.11 LABORATORIO: Operadores y expresiones

LABORATORIO

Tiempo Estimado

15-20 minutos

Nivel de Dificultad

Fácil

Objetivos

- Mejorar la habilidad de implementar números, operadores y operaciones aritméticas en Python.
- Utilizar la función `print()` y sus capacidades de formato.
- Aprender a expresar fenómenos del día a día en términos de un lenguaje de programación.

Escenario

La tarea es preparar un código simple para evaluar o encontrar el tiempo final de un periodo de tiempo dado, expresándolo en horas y minutos. Las horas van de 0 a 23 y los minutos de 0 a 59. El resultado debe ser mostrado en la consola.

Por ejemplo, si el evento comienza a las 12:17 y dura 59 minutos, terminará a las 13:16.

No te preocupes si tu código no es perfecto, está bien si acepta una hora inválida, lo más importante es que el código produzca una salida correcta acorde a la entrada dada.

Prueba el código cuidadosamente. Pista: utilizar el operador `%` puede ser clave para el éxito.

Datos de Prueba

Entrada de muestra:

```
1 hour = int(input("Hora de inicio (horas): "))
2 mins = int(input("Minuto de inicio (minutos): "))
3 dura = int(input("Duración del evento (minutos): "))
4
5 total_minutos = mins + dura
6 final_hour = (hour + total_minutos // 60) % 24
7 final_mins = total_minutos % 60
8
9 print(final_hour, final_mins, sep=":")
```

Console >_

```
Hora de inicio (horas): 4
Minuto de inicio (minutos): 5
Duración del evento (minutos): 6
4:11
```

PrevNext

My Progress

edube.org/study/catalog/python-essentials-1-pcep-30-02-ready-spanish/my-progress

OPENEDG

Home Learning Certification Achievements

Study

My Progress

Python Essentials 1 | PCEP-30-02 Ready [Spanish]

Welcome to PE1	PE1: Module 1	PE1: Module 2	PE1: Module 3	PE1: Module 4	PE1 Summary Test	PCEP-30-0x Exam
About the Course ✓	Study Resources ✓ Quiz: 100% Test: 100%	Study Resources ✓ Quiz: 100% Test: 100%	Study Resources ✓ Quiz: 100% Test: 100%	Study Resources ✓ Quiz: 92% Test: 91%	FINAL TEST: 97%	

Last activity - 2026-09-24 05:07:52

Resume Course